

SAEP

🕒 Criado em	26 de maio de 2026 11:17
☰ Tags	

Questões

- Etapa - 1 - produtos -id, nome, valor(decimal), quantidade(int) e categoria
movimentacoes -id, date, tipo(varchar(10))(entrada/saída), quantidade,
id_produto ---- relação(1-n)
- Etapa - 2 - inserir 3 dados em cada tabela
- Etapa - 3 - relação de 1-n produtos - movimentacoes
- Etapa - 4 - nome da view - vw_estoque (quantidade * valor)
- Etapa - 5 - iniciar uma api node e conectar com seu database
- Etapa - 6 - listar todos os produtos através de uma rota get /produtos (api)
- Etapa - 7 - cadastrar novos produtos, contendo validações para valor,
quantidade e categoria (api)
- Etapa - 8 - Registrar entrada de produtos no estoque (api)
- Etapa - 9 - Listar o valor total por categoria do produto, considerando a
quantidade e o valor (api)
- Etapa - 10 - Listar todas as saídas de produtos, apresentadas em ordem
decrecente por data.
- Etapa - 11 - identificar produtos que tenham atingido os limites mínimos
(valor:0) e máximo (valor:100) estabelecidos para o estoque com
identificação do percentual de nível atingido.
- Etapa - 12 - listar movimentações de entrada e saída no período informado
(conforme data inicial e data final), para cada um dos 7 campos
obrigatórios, conforme ordem: nome do produto, unidade de medida, total
de entradas, total de saídas, saldo no período, valor total financeiro das
entradas e valor total financeiro das saídas.
- Etapa - 13 - Listar os produtos com maior volume de saída no período
informado (conforme data inicial e data final) com os 3 campos

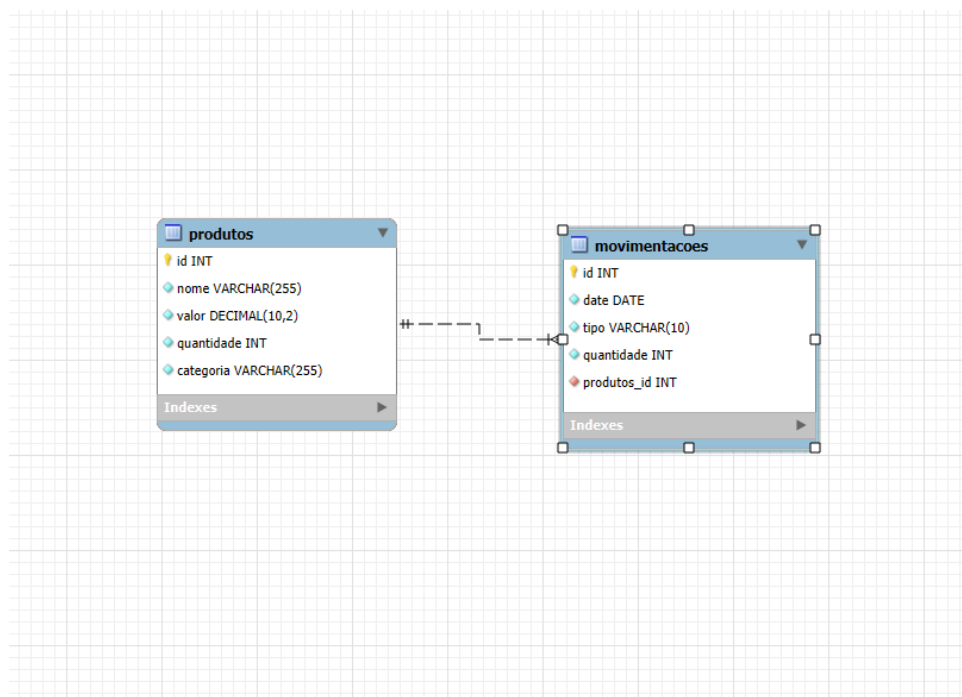
obrigatórios, conforme ordem: nome do produto, quantidade total de saída e valor total financeiro das saídas.

VSCODE

- npm init
- npm i
- npm i express mysql2
- npm i dotenv
- npm i cors
- npm i nodemon
- .gitignore - node_modules
- folder routes
- criar rota produtos

Modelo

- Criar diagramas produtos (id, nome, quantidade, valor, categoria) e movimentações (datas, tipo, quantidade



- Reverter o modelo para ddl

SQL

```
-- 1. Apaga o banco antigo se ele existir para começarmos d
o zero absoluto
DROP DATABASE IF EXISTS `mydb`;

-- 2. Cria o banco de dados limpo
CREATE DATABASE `mydb` DEFAULT CHARACTER SET utf8;
USE `mydb`;

-- 3. Cria a tabela de produtos (repare que adicionei a uni
dade_medida aqui)
CREATE TABLE `produtos` (
  `id` INT NOT NULL AUTO_INCREMENT,
  `nome` VARCHAR(255) NOT NULL,
  `valor` DECIMAL(10,2) NOT NULL,
  `quantidade` INT NOT NULL,
  `categoria` VARCHAR(255) NOT NULL,
  `unidade_medida` VARCHAR(50) NOT NULL, -- Coluna essencia
l para o seu exercício 12
  PRIMARY KEY (`id`)
) ENGINE = InnoDB;

-- 4. Cria a tabela de movimentações (repare que a coluna s
e chama 'datas')
CREATE TABLE `movimentacoes` (
  `id` INT NOT NULL AUTO_INCREMENT,
  `datas` DATE NOT NULL,
  `tipo` VARCHAR(10) NOT NULL, -- Onde vai entrar 'entrada'
ou 'saida'
  `quantidade` INT NOT NULL,
  `produtos_id` INT NOT NULL,
  PRIMARY KEY (`id`),
  CONSTRAINT `fk_movimentacoes_produtos`
    FOREIGN KEY (`produtos_id`)
    REFERENCES `produtos` (`id`)
) ENGINE = InnoDB;
```

```
-- 5. Inserindo dados de teste válidos para o ano de 2026
INSERT INTO produtos (nome, quantidade, valor, categoria, unidade_medida) VALUES
("Bodysplash", 12, 20.00, "Produto de beleza", "Litro"),
("Moletom", 10, 50.00, "Roupa", "Unidade"),
("Batom", 20, 10.00, "Produto de beleza", "Unidade");

INSERT INTO movimentacoes (datas, tipo, quantidade, produto_s_id) VALUES
('2026-05-10', 'entrada', 10, 1), -- 10 Bodysplash entrando
('2026-05-12', 'saida', 5, 1), -- 5 Bodysplash saindo
('2026-05-15', 'entrada', 20, 2), -- 20 Moletons entrando
('2026-05-20', 'saida', 10, 2), -- 10 Moletons saindo
('2026-05-22', 'saida', 4, 3); -- 4 Batons saindo
```

DDL

```
-- MySQL Script generated by MySQL Workbench
-- Tue May 26 14:46:14 2026
-- Model: New Model      Version: 1.0
-- MySQL Workbench Forward Engineering

SET @OLD_UNIQUE_CHECKS=@@UNIQUE_CHECKS, UNIQUE_CHECKS=0;
SET @OLD_FOREIGN_KEY_CHECKS=@@FOREIGN_KEY_CHECKS, FOREIGN_KEY_CHECKS=0;
SET @OLD_SQL_MODE=@@SQL_MODE, SQL_MODE='ONLY_FULL_GROUP_BY, STRICT_TRANS_TABLES, NO_ZERO_IN_DATE, NO_ZERO_DATE, ERROR_FOR_DIVISION_BY_ZERO, NO_ENGINE_SUBSTITUTION';

--
-- -----
-- Schema mydb
--
--
--
--
-- Schema mydb
--
--
--
CREATE SCHEMA IF NOT EXISTS `mydb` DEFAULT CHARACTER SET utf8 ;
```

```

USE `mydb` ;

-- -----
-- Table `mydb`.`produtos`
-- -----

CREATE TABLE IF NOT EXISTS `mydb`.`produtos` (
  `id` INT NOT NULL AUTO_INCREMENT,
  `nome` VARCHAR(255) NOT NULL,
  `valor` DECIMAL(10,2) NOT NULL,
  `quantidade` INT NOT NULL,
  `categoria` VARCHAR(255) NOT NULL,
  PRIMARY KEY (`id`))
ENGINE = InnoDB;

-- -----
-- Table `mydb`.`movimentacoes`
-- -----

CREATE TABLE IF NOT EXISTS `mydb`.`movimentacoes` (
  `id` INT NOT NULL AUTO_INCREMENT,
  `datas` DATE NOT NULL,
  `tipo` VARCHAR(10) NOT NULL,
  `quantidade` INT NOT NULL,
  `produtos_id` INT NOT NULL,
  PRIMARY KEY (`id`),
  INDEX `fk_movimentacoes_produtos_idx` (`produtos_id` ASC)
  VISIBLE,
  CONSTRAINT `fk_movimentacoes_produtos`
    FOREIGN KEY (`produtos_id`)
    REFERENCES `mydb`.`produtos` (`id`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION)
ENGINE = InnoDB;

SET SQL_MODE=@OLD_SQL_MODE;
SET FOREIGN_KEY_CHECKS=@OLD_FOREIGN_KEY_CHECKS;
SET UNIQUE_CHECKS=@OLD_UNIQUE_CHECKS;

```

DML

```
DROP DATABASE mydb;
USE mydb;

INSERT INTO produtos (nome, quantidade, valor, categoria)
VALUES ("Bodysplash", 120, 20.00, "Produto de beleza"),
("Moletom", 10, 50.00, "Roupa"),
("Batom", 20, 10.00, "Produto de beleza");

SELECT * FROM produtos;

INSERT INTO movimentacoes(datas, tipo, quantidade, produtos
_id)
VALUES ('2026-09-10', "saida", 10, 1),
('2026-04-12', "entrada", 20, 2),
('2026-05-09', "saida", 9, 3);

SELECT * FROM movimentacoes;

CREATE VIEW vw_produtos AS
SELECT nome, quantidade * valor, categoria
FROM produtos;

SELECT * FROM vw_produtos;
```

Server.js

```
const app = require('./app');
const dotenv = require('dotenv');

dotenv.config();

const port = process.env.PORT || 3000;

app.listen(port, () => {
  console.log(`Servidor rodando em http://localhost:${por
```

```
t}`);  
});
```

db.js

```
const mysql = require('mysql2');  
const dotenv = require('dotenv');  
const { connect } = require('node:http2');  
  
dotenv.config();  
  
const connection = mysql.createPool({  
  host:process.env.DB_HOST,  
  user:process.env.DB_USER,  
  password:process.env.DB_PASSWORD,  
  database:process.env.DB_NAME,  
  user:process.env.DB_USER,  
  port:process.env.DB_PORT  
});  
  
connection.getConnection((err) => {  
  if (err) {  
    console.error('Erro ao conectar ao banco de dados:', err);  
    return  
  } else {  
    console.log('Conexão com o banco de dados estabelecida com sucesso!');  
  }  
});  
  
module.exports = connection;
```

app.js

```

const express = require('express');
const cors = require('cors');
const dotenv = require('dotenv');
const connection = require('./db');
const app = express();

dotenv.config();

app.use(express.json());

const produtosRoutes = require ('./routes/produtos');
app.use ('/produtos', produtosRoutes);

module.exports = app;

```

.ENV

```

DB_HOST=localhost
DB_USER=root
DB_PASSWORD=root
DB_DATABASE=mydb
DB_PORT=3307

```

.gitignore

```

node_modules/

```

Rotas

- colocar acima:

```

const express = require('express');
const router = express.Router();
const db = require('./db');

```

6-) Listar todos os produtos


```

router.get('/listar', (req, res) => {
  db.query('SELECT * FROM produtos', (err, results) => {
    if (err) {
      return res.status(500).json({ error: 'Erro ao buscar produtos' });
    }
    res.json(results);
  });
});

```

7-) Cadastrar novos produtos

```

router.post('/cadastrar', (req, res) => {
  const { nome, quantidade, valor, categoria } = req.body;

  if (!nome || quantidade == null || valor == null || !categoria) {
    return res.status(400).json({ error: 'Payload inválido. Campos obrigatórios: nome, quantidade, valor, categoria' });
  }

  db.query(
    'INSERT INTO produtos (nome, valor, quantidade, categoria) VALUES (?, ?, ?, ?)',
    [nome, valor, quantidade, categoria],
    (err, results) => {
      if (err) {
        return res.status(500).json({ error: 'Erro ao cadastrar produto' });
      }
      return res.status(201).json({ message: 'Produto cadastrado com sucesso', id: results.insertId });
    }
  );
});

```

8-) Registrar entrada de produtos no estoque

```

router.post('/entrada', (req, res) => {
  const { id, quantidade } = req.body;
  if (!id || quantidade == null) {
    return res.status(400).json({ error: 'Deu erro' });
  }
  db.query(
    'UPDATE produtos SET quantidade = quantidade + ? WHERE id = ?',
    [quantidade, id],
    (err, results) => {
      if (err) {
        return res.status(500).json({ error: 'Erro ao registrar entrada de produto' });
      }
      if (results.affectedRows === 0) {
        return res.status(404).json({ error: 'Produto não encontrado' });
      }
      return res.status(200).json({ message: 'Entrada de produto registrada com sucesso' });
    }
  );
});

```

9-) Listar o valor total por categoria do produto, considerando a quantidade e o valor

```

router.get('/valor-total', (req, res) => {
  db.query('SELECT categoria, SUM(quantidade * valor) as valor_total FROM produtos GROUP BY categoria', (err, results) => {
    if (err) {
      return res.status(500).json({ error: 'Erro ao buscar valor total por categoria' });
    }
    res.json(results);
  });
});

```

```
});
});
```

10-) listar todas as saídas de produtos, apresentados em ordem decrescente por data

```
router.get('/saidas', (req, res) => {
  // Trocado de `date` para `datas`
  db.query('SELECT * FROM movimentacoes WHERE tipo = "saida" ORDER BY datas DESC', (err, results) => {
    if (err) {
      console.error(err); // Deixei isso para você ver erros futuros no terminal
      return res.status(500).json({ error: 'Erro ao buscar saídas de produtos' });
    }
    res.json(results);
  });
});
```

11-) Identificar produtos que tenham atingido os limites mínimos (valor:0) e máximo (valor:100) estabelecidos para o estoque com identificação do percentual de nível atingido

```
router.get('/limites', (req, res) => {
  db.query('SELECT id, nome, quantidade, valor, categoria, (quantidade / 100) * 100 AS percentual FROM produtos WHERE quantidade <= 0 OR quantidade >= 100', (err, results) => {
    if (err) {
      return res.status(500).json({ error: 'Erro ao buscar produtos com limites atingidos' });
    }
    res.json(results);
  });
});
```

12-) Listar movimentações de entrada e saída no período informado (conforme data inicial e data final), para cada um dos 7 campos obrigatórios, conforme ordem: nome do produto, unidade de medida, total de entradas, total de saídas, saída no período, valor total financeiro das entradas e valor financeiro das saídas

```
router.get('/movimentacoes', (req, res) => {
  const { data_inicial, data_final } = req.query;
  if (!data_inicial || !data_final) {
    return res.status(400).json({ error: 'Deu erro' });
  }
  db.query('SELECT * FROM movimentacoes WHERE `date` BETWEEN ? AND ?', [data_inicial, data_final], (err, results) => {
    if (err) {
      return res.status(500).json({ error: 'Erro ao buscar movimentações' });
    }
    res.json(results);
  });
});
```

// 12- Listar movimentações de entrada e saída no período informado

```
router.get('/movimentacoes', (req, res) => {
  const { data_inicial, data_final } = req.query;

  if (!data_inicial || !data_final) {
    return res.status(400).json({ error: 'Por favor, informe a data_inicial e data_final na URL.' });
  }
}
```

// Query robusta para trazer exatamente os 7 campos obrigatórios na ordem solicitada

```
const querySQL = `
  SELECT
    p.nome AS nome_do_produto,
    p.unidade_medida,
```

```

        SUM(CASE WHEN m.tipo = 'entrada' THEN m.quantid
ade ELSE 0 END) AS total_de_entradas,
        SUM(CASE WHEN m.tipo = 'saida' THEN m.quantidad
e ELSE 0 END) AS total_de_saidas,
        SUM(CASE WHEN m.tipo = 'saida' THEN m.quantidad
e ELSE 0 END) AS saida_no_periodo,
        SUM(CASE WHEN m.tipo = 'entrada' THEN m.quantid
ade * p.valor ELSE 0 END) AS valor_total_financeiro_das_ent
radas,
        SUM(CASE WHEN m.tipo = 'saida' THEN m.quantidad
e * p.valor ELSE 0 END) AS valor_financeiro_das_saidas
    FROM produtos p
    INNER JOIN movimentacoes m ON p.id = m.produtos_id
    WHERE m.datas BETWEEN ? AND ?
    GROUP BY p.id, p.nome, p.unidade_medida;
`;

db.query(querySQL, [data_inicial, data_final], (err, re
sults) => {
    if (err) {
        console.error('Erro no SQL:', err); // Ajuda vo
cê a debugar se algo falhar
        return res.status(500).json({ error: 'Erro ao b
uscar movimentações no período' });
    }
    res.json(results);
});
});

```

13-) Listar os produtos com maior volume de saída no período informado (conforme data inicial e data final) com os 3 campos obrigatórios, conforme ordem: nome do produto, quantidade total de saídas e valor total financeiro das saídas

```

router.get('/maiores-saidas', (req, res) => {
    const { data_inicial, data_final } = req.query;
    if (!data_inicial || !data_final) {
        return res.status(400).json({ error: 'Deu erro' });
    }

```

```

    }
    db.query('SELECT * FROM movimentacoes WHERE `date` BETWEEN ? AND ?', [data_inicial, data_final], (err, results) => {
        if (err) {
            return res.status(500).json({ error: 'Erro ao buscar movimentações' });
        }
        res.json(results);
    });
});

router.get('/maiores-saidas', (req, res) => {
    const { data_inicial, data_final } = req.query;

    // Validação para garantir que as datas foram enviadas na URL
    if (!data_inicial || !data_final) {
        return res.status(400).json({ error: 'Por favor, informe data_inicial e data_final na URL.' });
    }

    // Query SQL estruturada para trazer os 3 campos obrigatórios na ordem exata
    const querySQL = `
        SELECT
            p.nome AS nome_do_produto,
-- [Campo 1]
            SUM(m.quantidade) AS quantidade_total_de_saidas,
-- [Campo 2]
            SUM(m.quantidade * p.valor) AS valor_total_financeiro_das_saidas -- [Campo 3]
        FROM produtos p
        INNER JOIN movimentacoes m ON p.id = m.produtos_id
        WHERE m.tipo = 'saida' AND m.datas BETWEEN ? AND ?
-- Filtra apenas as SAÍDAS dentro do período
        GROUP BY p.id, p.nome
        ORDER BY quantidade_total_de_saidas DESC;
    `;

```

```

-- Garante os de maior volume primeiro
    `;

    db.query(querySQL, [data_inicial, data_final], (err, results) => {
        if (err) {
            console.error("ERRO NO BANCO (ROTA 13) ->", err); // Printa o erro real no terminal do VS Code
            return res.status(500).json({ error: 'Erro ao buscar produtos com maiores saídas' });
        }
        res.json(results); // Retorna a lista ordenada para o Postman
    });
});

module.exports = router;

```

- Colocar após todas as rotas criadas:

```

module.exports = router;

```

API

6-)

GET http://localhost:3000/produtos/listar Send

Docs Params Authorization Headers (6) Body Scripts Settings Cookies

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers (8) Test Results 200 OK - 19 ms - 529 B Save Response

JSON Preview Visualize

```

1 [
2   {
3     "id": 1,
4     "nome": "BodySplash",
5     "valor": "20.00",
6     "quantidade": 12,
7     "categoria": "Produto de beleza"
8   },
9   {
10    "id": 2,
11    "nome": "Moletom",
12    "valor": "60.00",
13    "quantidade": 10,
14    "categoria": "Roupa"
15  },
16  {
17    "id": 3,
18    "nome": "Batom",

```

7-)

saep / cadastrar Save

POST http://localhost:3000/produtos/cadastrar Send

Docs Params Authorization Headers (8) Body Scripts Settings Cookies

none form-data x-www-form-urlencoded raw binary GraphQL JSON Schema Beautify

```

1 {
2   "nome": "Shampoo",
3   "quantidade": 19,
4   "valor": 23.00,
5   "categoria": "produto de cabelo"
6 }

```

Body Cookies Headers (8) Test Results 201 Created - 55 ms - 323 B Save Response

JSON Preview Visualize

```

1 {
2   "message": "Produto cadastrado com sucesso",
3   "id": 4
4 }

```

8-)

saep / entrada

POST http://localhost:3000/produtos/entrada

Docs Params Authorization Headers (8) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

Schema Beautify

```
1 {
2   "id": 1,
3   "quantidade": 10
4 }
5
```

Body Cookies Headers (8) Test Results

200 OK - 12 ms - 322 B

Save Response

JSON Preview Visualize

```
1 {
2   "message": "Entrada de produto registrada com sucesso"
3 }
```

9-)

GET http://localhost:3000/produtos/valor-total

Docs Params Authorization Headers (6) Body Scripts Settings

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers (8) Test Results

200 OK - 7 ms - 371 B

Save Response

JSON Preview Visualize

	categoria	valor_total
0	Produto de beleza	440.00
1	Roupa	500.00

10-)

saep / saidas

GET http://localhost:3000/produtos/saidas

Send

Docs Params Authorization Headers (6) Body Scripts Settings

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers (8) Test Results

200 OK • 28 ms • 450 B

Save Response

JSON Preview Visualize

```

1  [
2    {
3      "id": 1,
4      "datas": "2026-09-18T03:00:00.000Z",
5      "tipo": "saida",
6      "quantidade": 10,
7      "produtos_id": 1
8    },
9    {
10     "id": 3,
11     "datas": "2026-06-09T03:00:00.000Z",
12     "tipo": "saida",
13     "quantidade": 9,
14     "produtos_id": 3
15   }
16 ]

```

11-)

saep / limites

GET http://localhost:3000/produtos/limites

Send

Docs Params Authorization Headers (6) Body Scripts Settings

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers (8) Test Results

200 OK • 19 ms • 387 B

Save Response

JSON Preview Visualize

```

1  [
2    {
3      "id": 1,
4      "nome": "Bodysplash",
5      "quantidade": 120,
6      "valor": "20.00",
7      "categoria": "Produto de beleza",
8      "percentual": "120.0000"
9    }
10 ]

```

12-)

- http://localhost:3000/produtos/movimentacoes?data_inicial=2026-05-01&data_final=2026-05-31

saep / **datas** Save Share

GET ▼ http://localhost:3000/produtos/movimentacoes?data_inicial=2026-05-01&data_final=2026-05-31 Send

Docs Params Authorization Headers (6) Body Scripts Settings Cook

Query Params

☒ Key	Value	Description	...	Bulk Ed
☒ data_inicial	2026-05-01			
☒ data_final	2026-05-31			
Key	Value	Description		

Body Cookies Headers (8) Test Results 200 OK 19 ms 907 B Save Response

{} JSON ▼ Preview Visualize ▼

```

12  {
13    "nome_do_produto": "Moletom",
14    "unidade_medida": "Unidade",
15    "total_de_entradas": "28",
16    "total_de_saidas": "18",
17    "saida_no_periodo": "18",
18    "valor_total_financeiro_das_entradas": "1800.00",
19    "valor_financeiro_das_saidas": "500.00"
20  },
21  {
22    "nome_do_produto": "Batom",
23    "unidade_medida": "Unidade",
24    "total_de_entradas": "8",
25    "total_de_saidas": "4",
26    "saida_no_periodo": "4",
27    "valor_total_financeiro_das_entradas": "8.00",
28    "valor_financeiro_das_saidas": "40.00"
29  }
30 }

```

13-)

- http://localhost:3000/produtos/maiores-saidas?data_inicial=2026-05-01&data_final=2026-05-31

saep / **maiores-saidas** Save Share

GET ▼ http://localhost:3000/produtos/maiores-saidas?data_inicial=2026-05-01&data_final=2026-05-31 Send

Docs Params Authorization Headers (6) Body Scripts Settings Cookies

Query Params

☒ Key	Value	Description	...	Bulk Edit
☒ data_inicial	2026-05-01			
☒ data_final	2026-05-31			
Key	Value	Description		

Body Cookies Headers (8) Test Results 200 OK 30 ms 595 B Save Response

{} JSON ▼ Preview Visualize ▼

```

1  [
2    {
3      "nome_do_produto": "Moletom",
4      "quantidade_total_de_saidas": "18",
5      "valor_total_financeiro_das_saidas": "500.00"
6    },
7    {
8      "nome_do_produto": "BodySplash",
9      "quantidade_total_de_saidas": "5",
10     "valor_total_financeiro_das_saidas": "100.00"
11   },
12   {
13     "nome_do_produto": "Batom",
14     "quantidade_total_de_saidas": "4",
15     "valor_total_financeiro_das_saidas": "40.00"
16   }
17 ]

```

Como fazer funcionar

Para fazer uma requisição de API do tipo **POST** funcionar, você precisa **enviar os dados no formato correto para o endereço (endpoint) da API e garantir que a sua aplicação ou servidor esteja configurado para processá-los**. [1, 2, 3, 4]

Abaixo está o passo a passo para estruturar e testar o seu POST com sucesso:

1. Configure o seu Testador (Ex: Postman ou Insomnia)

Ferramentas como Postman ou Insomnia são essenciais para simular envios e verificar erros. [5, 6, 7]

- **Método:** Selecione **POST**.
- **URL:** Insira o endereço completo do endpoint da sua API (ex: `https://suaapi.com` ou `http://localhost:3000/api`). [7]

2. Defina os Cabeçalhos (Headers)

O cabeçalho informa à API como ela deve ler os dados que você está enviando. [4, 8]

- **Key:** `Content-Type`
- **Value:** `application/json` (ou `application/x-www-form-urlencoded`, dependendo do que o backend exige).

3. Envie os Dados no Corpo (Body)

Diferente do método GET, o POST envia informações dentro do "corpo" da requisição. [2, 4]

- Na aba **Body**, escolha a opção **raw** e selecione o formato **JSON**.
- Escreva os dados em formato de chaves e valores, por exemplo: [2]

```
{  
  "nome": "João Silva",  
  "idade": 28,  
  "email": "joao@email.com"  
}
```

4. Verifique a Resposta (Response)

Ao clicar em **Send**, a API responderá com um código de status HTTP: [6, 9]

- **201 Created:** Sucesso! O dado foi criado no servidor.
- **400 Bad Request:** Dados enviados de forma incorreta ou faltando campos obrigatórios.
- **401 Unauthorized:** Falta de autenticação (precisa de token).
- **404 Not Found:** URL ou endpoint digitado incorretamente.
- **500 Internal Server Error:** Erro no código do servidor (backend). [4, 10, 11, 12, 13]

Para que eu possa te ajudar de forma mais precisa, me diga:

- Você está **desenvolvendo a API** (backend) ou apenas **consumindo** (enviando dados de um site/app)?
- Qual **linguagem ou framework** você está usando (Node.js, Python, PHP, React, etc.)?
- Qual erro específico você está recebendo? [4, 6, 7, 8, 9, 14, 15]

Fazer uma API GET funcionar envolve dois lados: o **servidor** (que cria e fornece os dados) e o **cliente** (que faz a requisição e consome os dados). O método GET serve exclusivamente para **buscar informações** sem alterá-las. [1, 2, 3, 4, 5]

Siga este passo a passo para estruturar e testar a sua API:

1. Criando a API no Servidor (Exemplo com Node.js e Express)

Se você está criando o servidor do zero, precisa definir a rota e o que ela vai responder. [6]

1. No seu terminal, inicie o projeto e instale o Express:

```
npm init -y
npm install express
```

2. Crie um arquivo `index.js` e configure o método GET:

```
const express = require('express');
const app = express();
const port = 3000;

// Criando a rota GET
app.get('/api/dados', (req, res) => {
```

```

    res.json({
      mensagem: "Sucesso! Sua API GET está funcionando.",
      status: "online"
    });
  });
});

// Iniciando o servidor
app.listen(port, () => {
  console.log(`Servidor rodando em http://localhost:${port}`);
});

```

3. Execute no terminal: `node index.js`. [7, 8]

2. Testando e Consumindo a API

Antes de colocar no seu aplicativo ou site, é recomendado testar a rota diretamente. [9]

- **Pelo Navegador:** Basta abrir o navegador e digitar a URL da sua requisição (ex: `http://localhost:3000/api/dados`). Como o GET é o método padrão do navegador, ele mostrará o JSON na tela. [9, 10, 11, 12]
- **Por Ferramentas de Teste (Recomendado):** Para testar de forma profissional (verificando status code 200, cabeçalhos, etc.), use o Postman ou o Insomnia. Basta criar uma nova requisição **GET**, colar a URL e clicar em *Send*. [13, 14, 15]

3. Consumindo no Front-End (Exemplo com JavaScript/Fetch)

Para fazer o seu site/app buscar esses dados, use o método nativo `fetch()`: [16]

```

fetch('http://localhost:3000/api/dados')
  .then(response => response.json()) // Converte a resposta para JSON
  .then(data => {
    console.log(data); // Exibe os dados no console
  })
  .catch(error => {
    console.error('Erro ao buscar dados:', error);
  });

```

Onde você está encontrando problemas?

Para eu te ajudar com um código mais específico, me diga:

1. **Onde você está criando a API?** (Node.js, Java Spring Boot, Python, PHP, C#)

2. **Qual é o erro que aparece?** (Ex: *404 Not Found*, *403 Forbidden*, *500 Internal Server Error*)
3. **Qual é o seu objetivo final?** (Ex: Conectar com um banco de dados, integrar com um sistema externo) [6, 15, 17, 18]

To create a table in MySQL, use the `CREATE TABLE` statement to define your table name, column structures, and data restrictions. [1, 2]

Basic Syntax

```
CREATE TABLE [IF NOT EXISTS] table_name (  
    column1_name datatype constraints,  
    column2_name datatype constraints,  
    ...  
    PRIMARY KEY (column_name)  
);
```

- `IF NOT EXISTS`: Prevents an error if the table name is already used.
- `datatype`: Defines the type of data allowed (e.g., `INT`, `VARCHAR`, `DATE`).
- `constraints`: Set rules for data rows (e.g., `NOT NULL`, `UNIQUE`, `AUTO_INCREMENT`). [1, 2, 3, 4]

Practical Example

Here is how to create a typical `users` database table:

```
CREATE TABLE IF NOT EXISTS users (  
    id INT AUTO_INCREMENT,  
    username VARCHAR(50) NOT NULL,  
    email VARCHAR(100) UNIQUE,  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    PRIMARY KEY (id)  
);
```

Common Column Rules

- `NOT NULL`: Forces the column to always require a value.
- `AUTO_INCREMENT`: Automatically adds 1 to the integer value for every new entry.
- `DEFAULT`: Automatically applies a backup value if no value is entered.
- `PRIMARY KEY`: Chooses the main column used to uniquely identify each row. [4, 5, 6, 7, 8]

Next Step Execution

Before writing this query, make sure to explicitly pick your target database first: [9]

```
USE your_database_name;
```

If you want to view your new table layout after running the query, use the [MySQL Reference Documentation](#) suggestion to check your work: [10]

```
DESCRIBE users;
```

Tell me what **kind of data** you want to store (e.g., store products, blog posts, student profiles) so I can write the **exact SQL code** you need.

To create a view in MySQL, use the `CREATE VIEW` statement followed by the view name and the `SELECT` query that defines it. [1, 2]

Syntax

```
CREATE [OR REPLACE] VIEW view_name AS
SELECT column1, column2, ...
FROM table_name
WHERE condition;
```

The `OR REPLACE` clause is optional but highly useful; it overwrites the view if it already exists. [3, 4]

Examples

1. Simple View (Filtering Rows) This creates a virtual table containing only the name and contact details of customers based in Brazil: [1, 5, 6, 7]

```
CREATE OR REPLACE VIEW brazil_customers_vw AS
SELECT CustomerName, ContactName
FROM Customers
WHERE Country = 'Brazil';
```

2. Complex View (Joining Multiple Tables) Views can encapsulate complex logic like `INNER JOIN` operations so you do not have to rewrite them every time: [2, 6]

```
CREATE OR REPLACE VIEW order_summary_vw AS
SELECT o.OrderID, c.CustomerName, o.OrderDate
FROM Orders o
INNER JOIN Customers c ON o.CustomerID = c.CustomerID;
```

How to Use the View

Once created, you can query a view exactly like a normal database table: [2, 8]

```
SELECT * FROM brazil_customers_vw;
```

Managing Views

- **Modify a View:** Use `ALTER VIEW view_name AS SELECT ...` or simply use `CREATE OR REPLACE VIEW`.
- **Delete a View:** Use `DROP VIEW view_name;`. [1, 3, 5, 9]

For deeper configuration rules regarding advanced security contexts and algorithms, check the official [MySQL CREATE VIEW Documentation](#). [3, 10, 11]

If you would like, tell me:

- What **columns** and **tables** you want to include
- Any specific **filters** or **joins** you need

I can write the exact SQL query tailored to your database schema. [5]

In MySQL, the `GROUP BY` clause must always be placed before the `ORDER BY` clause in your query structure. While `GROUP BY` collapses rows with matching values into summary rows (usually paired with aggregate functions like `COUNT()` or `SUM()`), `ORDER BY` sorts the final aggregated results. [1, 2, 3]

Standard Query Syntax

```
SELECT column_name, COUNT(*) as total
FROM table_name
WHERE condition
GROUP BY column_name
ORDER BY total DESC;
```

Concrete Examples

1. Sorting by an Aggregate Function

To find out how many customers are in each country, and list the countries with the most customers first, sort by the calculated `COUNT()`: [3]

```
SELECT Country, COUNT(CustomerID) AS NumberOfCustomers
FROM Customers
GROUP BY Country
ORDER BY COUNT(CustomerID) DESC;
```

2. Sorting by the Grouped Column

To group rows by status and sort the resulting summary alphabetically by that status name: [4, 5]

```
SELECT status, SUM(quantity) AS TotalQuantity
FROM orders
GROUP BY status
ORDER BY status ASC;
```

How MySQL Processes Both Clauses Internally

The database evaluates clauses in a fixed sequence: [6, 7]

1. **FROM**: Identifies the target table.
 2. **WHERE**: Filters individual rows before grouping.
 3. **GROUP BY**: Groups identical values together and calculates aggregates.
 4. **HAVING**: Filters the grouped summary records.
 5. **SELECT**: Determines which columns/aggregates to display.
 6. **ORDER BY**: Sorts the final output rows. [2, 6]
-

Common Pitfalls to Avoid

- **Ordering Before Grouping:** Trying to write **ORDER BY** before **GROUP BY** will trigger a syntax error. [2, 8]
- **Strict SQL Modes (**ONLY_FULL_GROUP_BY**):** Modern MySQL engines enforce strict SQL behavior. If you include a column in your **SELECT** or **ORDER BY** list that isn't inside an aggregate function (like **MAX()**) or named in the **GROUP BY** clause, MySQL will throw an error. [9, 10, 11]
- **Implicit Sorting:** In older legacy versions (MySQL 5.7 and earlier), **GROUP BY** implicitly sorted results. This feature has been completely removed in MySQL 8.0. You **must** explicitly use **ORDER BY** if you want your rows sorted. [12, 13]

Would you like help writing a specific query for your database, or are you trying to resolve a **particular MySQL error message**?

In MySQL, you can use mathematical expressions inside both the `GROUP BY` and `ORDER BY` clauses to group or sort your data dynamically. [1, 2]

The strict logical evaluation sequence for your query must always follow this order: `SELECT` $\rightarrow$ `FROM` $\rightarrow$ `WHERE` $\rightarrow$ `GROUP BY` $\rightarrow$ `HAVING` $\rightarrow$ `ORDER BY`. [3]

1. Math in `GROUP BY`

You can group rows by the result of a mathematical expression (e.g., dividing ranges, rounding numbers, or bucketing ages). [2, 4, 5, 6, 7]

```
SELECT
  FLOOR(price / 10) * 10 AS price_range,
  COUNT(*) AS total_items
FROM products
GROUP BY FLOOR(price / 10) * 10;
```

- **What it does:** Groups products into \$10 price intervals (e.g., 0-9, 10-19) using floor division and counts the items in each bucket. [2, 7]

2. Math in `ORDER BY`

You can sort data using calculated math formulas, such as calculating margins, differences, or custom weighted scores. [1, 8, 9, 10, 11]

```
SELECT
  product_name,
  retail_price,
  wholesale_cost
FROM products
ORDER BY (retail_price - wholesale_cost) DESC;
```

- **What it does:** Dynamically calculates the profit margin for every row and sorts the results to show the most profitable items first. [1]

3. Combining Both with Aggregate Math

When you combine both clauses, `ORDER BY` executes **after** `GROUP BY`. Therefore, any math in `ORDER BY` must use aggregate functions (like `SUM()`, `AVG()`, `COUNT()`) or columns present in the `GROUP BY` clause. [7, 12, 13]

```
SELECT
  category_id,
  SUM(stock_quantity) AS total_stock,
  AVG(price) AS average_price
FROM products
GROUP BY category_id
ORDER BY (SUM(stock_quantity) * AVG(price)) DESC;
```

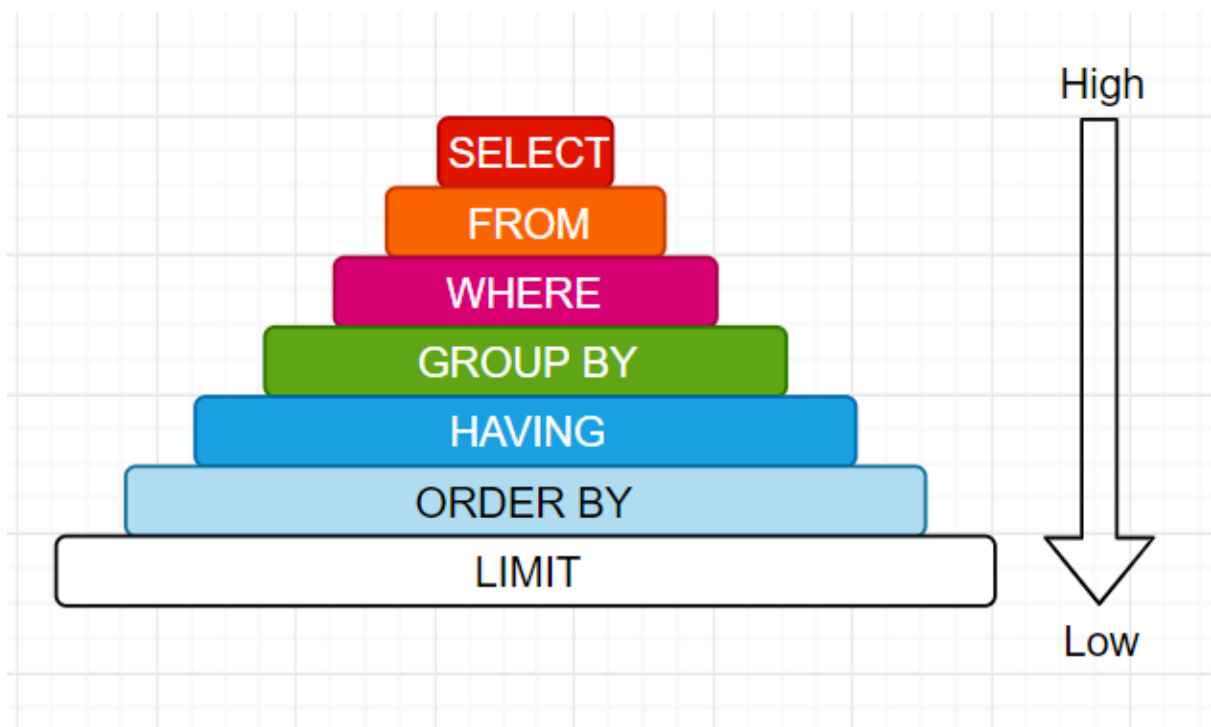
- **What it does:** Groups your items by category, computes the total stock and average price per group, and then sorts those grouped categories by their total calculated inventory value. [7, 13, 14, 15, 16]

Best Practices & Performance Tips

- **Use Aliases:** Calculate complex math once in the `SELECT` clause and reference the column alias name in the `ORDER BY` clause to make your query cleaner.
- **Index Warnings:** Doing raw math operations inside `GROUP BY` or `ORDER BY` prevents MySQL from utilizing standard column indexes. If your dataset is massive, consider caching the calculated value into a generated/virtual column and adding an index to it instead. [1, 11, 17, 18, 19]

If you would like to tailor this query, tell me:

- What do your **table structure** and columns look like?
- What specific **mathematical calculation** are you trying to achieve?



Erros

1-)

O seu código Node.js (Express) em si **está sintaticamente correto**. Se algo deu errado, é quase certeza que o problema não está nessa rota especificamente, mas sim na **comunicação com o banco de dados**, na **estrutura da tabela**, ou na forma como a **rota foi chamada**.

O suspeito número 1 em códigos em português é a coluna ``date``. Muitas vezes criamos a coluna no banco como `data`, mas no código chamamos de `date`, o que quebra a query. Outro suspeito é o valor `"saida"` (sem acento) na query vs `"saída"` (com acento) salvo no banco.

Para garantir que tudo funcione, aqui está o passo a passo completo, do banco de dados até o teste final.

Passo 1: MySQL Workbench (A Estrutura do Banco)

O erro mais comum acontece aqui: as colunas do banco não batem exatamente com as da sua query.

1. Abra o MySQL Workbench e conecte-se ao seu banco de dados.
2. Execute o seguinte comando para verificar a estrutura da sua tabela:SQL

```
DESCRIBE movimentacoes;
```

3. Verifique as colunas:

- Você tem uma coluna chamada exatamente `date`? Se ela se chamar `data` ou `data_saida`, a sua query no VS Code vai falhar.
- Você tem uma coluna chamada `tipo`?

4. Verifique os dados:

Execute `SELECT * FROM movimentacoes;`. O tipo está escrito exatamente como `saida` (sem acento)? O banco diferencia "saida" de "saída" dependendo da configuração (collation).

Script de Correção (Caso sua tabela esteja diferente):

Se quiser criar uma tabela de teste exata para o seu código, rode isto no Workbench:

SQL

```
CREATE TABLE movimentacoes (
  id INT AUTO_INCREMENT PRIMARY KEY,
  produto_nome VARCHAR(100),
  tipo VARCHAR(50), /* 'entrada' ou 'saida' */
  `date` DATETIME DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO movimentacoes (produto_nome, tipo, `date`) VALUES
('Notebook', 'saida', '2026-06-01 10:00:00'),
('Teclado', 'saida', '2026-06-01 11:30:00'),
('Mouse', 'entrada', '2026-06-01 09:00:00');
```

Passo 2: VS Code (O Servidor e a Rota)

Se o código for apenas o snippet que você mandou, ele precisa estar inserido em um servidor que inicie o Express e conecte no banco.

1. No VS Code, abra o terminal (`Ctrl` + ``` ou `Cmd` + ```) e garanta que instalou as dependências: Bash

```
npm install express mysql2
```

2. Verifique o seu arquivo principal (ex: `index.js` ou `app.js`). Ele precisa estar estruturado mais ou menos assim para a sua rota funcionar: JavaScript

```
const express = require('express');
const mysql = require('mysql2');

const app = express();
const router = express.Router();
app.use(express.json());

// Configuração da conexão com o banco de dados
const db = mysql.createConnection({
  host: 'localhost',
  user: 'root', // Seu usuário do MySQL
  password: 'sua_senha_aqui', // Sua senha do MySQL
  database: 'seu_banco_de_dados' // Nome do seu banco
});

db.connect((err) => {
  if (err) {
    console.error('Erro ao conectar ao banco de dados:', err);
    return;
  }
  console.log('Conectado ao MySQL com sucesso!');
```

```
});

// Sua rota exatamente como você escreveu
// 10- listar todas as saídas de produtos, apresentados em ordem decrescente por data
router.get('/saidas', (req, res) => {
  // CUIDADO: Se no seu banco a coluna for "data", mude `date` para data aqui em baixo!
  db.query('SELECT * FROM movimentacoes WHERE tipo = "saida" ORDER BY `date` DESC', (err, results) => {
    if (err) {
      console.error(err); // <-- Adicione isso para ver o erro real no terminal do VS Code
      return res.status(500).json({ error: 'Erro ao buscar saídas de produtos' });
    }
    res.json(results);
  });
});

// Define que as rotas vão usar um caminho base (ex: http://localhost:3000/api/saidas)
app.use('/api', router);

// Inicia o servidor
const PORT = 3000;
app.listen(PORT, () => {
  console.log(`Servidor rodando na porta ${PORT}`);
});
```

3. Rode o servidor no terminal:Bash

```
node index.js
```

(Você deve ver a mensagem "Conectado ao MySQL com sucesso!" e "Servidor rodando na porta 3000").

Passo 3: Postman (O Teste)

Muitas vezes o erro está na forma como fazemos a requisição.

1. Abra o Postman.
2. Crie uma nova requisição clicando no botão "+".
3. No método (o dropdown à esquerda), selecione **GET**.
4. Na barra de URL, digite o endereço correto. Se você seguiu o exemplo do VS Code acima, a URL será: Plaintext

```
http://localhost:3000/api/saidas
```

(Atenção: Se você não usou `app.use('/api', router)` no seu código, a URL será apenas `http://localhost:3000/saidas`).

5. Clique em **Send** (Enviar).

O que observar:

- Se retornar um array JSON vazio `[]`, a query funcionou, mas não há nenhuma linha no banco onde a coluna `tipo` seja exatamente igual a `"saida"`.
- Se retornar `Status 500` com a sua mensagem de erro, olhe imediatamente para o **Terminal do VS Code**; o `console.error(err)` vai te dizer exatamente qual foi o problema no SQL (ex: *Unknown column 'date' in 'order clause'*).

Qual erro exato está aparecendo para você no terminal do VS Code ou no retorno do Postman quando você tenta fazer a requisição?

2-)

Ah, agora o mistério está resolvido! Olhando o seu script SQL, eu encontrei exatamente onde está o curto-circuito entre o seu banco de dados (MySQL) e o seu servidor (VS Code).

Foi exatamente o que eu suspeitava: **os nomes e os dados não estão batendo**.

Aqui estão os três problemas exatos e como vamos resolver cada um deles.

1. O nome da coluna de data (O Erro 500)

No seu código do VS Code, a sua query pede para ordenar pela coluna `date`:

```
| ORDER BY `date` DESC`
```

Mas, olhando o seu script de criação da tabela `movimentacoes`, a sua coluna se chama `datas`:

```
| `datas` DATE NOT NULL
```

O que acontece: O Node vai bater no banco, o banco vai dizer "não conheço nenhuma coluna chamada `date`" e o seu código vai retornar aquele erro 500 do `catch`.

2. O valor do tipo (A lista vazia)

No seu código do VS Code, você está buscando estritamente onde o tipo é

```
"saida" :
```

```
| WHERE tipo = "saida"
```

Mas, nos seus `INSERT`s do MySQL, você colocou valores aleatórios para o tipo:

```
| VALUES ('2026-09-10', "sla", 10, 1), ('2026-04-12', "abc", 20, 2), ('2026-05-09',  
"dnd", 9, 3);
```

O que acontece: Mesmo que você arrume o erro da data, quando você rodar no Postman, ele vai te devolver uma lista vazia `[]` porque não existe nenhum registro com o tipo `"saida"`.

3. A ordem de execução do script SQL

No arquivo que você me mandou, você tentou dar um `DROP DATABASE`, `USE mydb;` e fazer os `INSERT`s **antes** de criar o banco e as tabelas (o código do *Workbench Forward Engineering* está lá no final). O MySQL vai dar erro logo na primeira linha porque você não pode inserir dados em uma tabela que ainda não foi criada.

Como consertar (O Passo a Passo)

Para deixar tudo perfeito e funcionando, faça as duas alterações abaixo:

No VS Code (Corrigir a Rota)

Mude a palavra `date` para `datas` na sua query SQL. O código correto fica assim:
JavaScript

```
// 10- listar todas as saídas de produtos, apresentados em ordem decrescente por data  
router.get('/saidas', (req, res) => {  
  // Trocado de `date` para `datas`  
  db.query('SELECT * FROM movimentacoes WHERE tipo = "saida" ORDER BY datas DESC',  
(err, results) => {  
    if (err) {  
      console.error(err); // Deixei isso para você ver erros futuros no terminal  
      return res.status(500).json({ error: 'Erro ao buscar saídas de produtos' });  
    }  
    res.json(results);  
  });  
});
```

No MySQL Workbench (Corrigir a Ordem e os Dados)

Rode o script de criação **primeiro** e, só depois, insira os dados de teste usando a palavra **"saida"**. Apague o que você rodou antes e execute este script na ordem certa:

SQL

```
-- 1. PRIMEIRO VOCÊ CRIA O BANCO E AS TABELAS
CREATE SCHEMA IF NOT EXISTS `mydb` DEFAULT CHARACTER SET utf8 ;
USE `mydb` ;

CREATE TABLE IF NOT EXISTS `mydb`.`produtos` (
  `id` INT NOT NULL AUTO_INCREMENT,
  `nome` VARCHAR(255) NOT NULL,
  `valor` DECIMAL(10,2) NOT NULL,
  `quantidade` INT NOT NULL,
  `categoria` VARCHAR(255) NOT NULL,
  PRIMARY KEY (`id`))
ENGINE = InnoDB;

CREATE TABLE IF NOT EXISTS `mydb`.`movimentacoes` (
  `id` INT NOT NULL AUTO_INCREMENT,
  `datas` DATE NOT NULL,
  `tipo` VARCHAR(10) NOT NULL, -- Certifique-se que cabe a palavra 'saida' (tem 5 letras, cabe no VARCHAR(10))
  `quantidade` INT NOT NULL,
  `produtos_id` INT NOT NULL,
  PRIMARY KEY (`id`),
  INDEX `fk_movimentacoes_produtos_idx` (`produtos_id` ASC) VISIBLE,
  CONSTRAINT `fk_movimentacoes_produtos`
    FOREIGN KEY (`produtos_id`)
      REFERENCES `mydb`.`produtos` (`id`)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION)
ENGINE = InnoDB;

-- 2. DEPOIS VOCÊ INSERE OS DADOS (Com a palavra 'saida' para poder testar)
INSERT INTO produtos (nome, quantidade, valor, categoria)
VALUES
("Bodysplash", 12, 20.00, "Produto de beleza"),
("Moletom", 10, 50.00, "Roupa"),
("Batom", 20, 10.00, "Produto de beleza");

INSERT INTO movimentacoes(datas, tipo, quantidade, produtos_id)
VALUES
('2026-09-10', 'saida', 10, 1),
('2026-04-12', 'entrada', 20, 2),
('2026-05-09', 'saida', 9, 3);
```

Após fazer essas duas alterações, reinicie o seu servidor Node (`Ctrl+C` e depois `node seu_arquivo.js`) e teste no Postman novamente. Vai funcionar direitinho!

Você quer que eu te ajude a montar a rota para listar as **entradas** seguindo essa mesma lógica corrigida?

3-)

O código da sua rota está **perfeito** e não vai causar nenhum erro no seu servidor Express. A sintaxe está correta e a lógica do banco de dados bate com a estrutura da tabela `produtos` que você me enviou antes.

Porém, se você testar isso no Postman agora mesmo, pode ter a falsa impressão de que deu errado.

Aqui está o que você precisa saber sobre essa rota específica:

1. O "Problema" dos Dados Atuais (Retorno Vazio)

A sua condição `WHERE quantidade <= 0 OR quantidade >= 100` está certíssima para buscar os produtos em situação crítica de estoque. O detalhe é que os produtos que você cadastrou no seu script anterior têm as seguintes quantidades: **12, 10 e 20**.

Como nenhum deles esgotou (`<= 0`) e nenhum está superlotado (`>= 100`), o Postman vai te retornar uma lista vazia `[]`. Isso não é um erro; o banco só está respondendo que, no momento, o estoque está dentro dos limites.

Como ver a rota funcionando de verdade:

Para testar no Postman e ver os dados aparecendo, insira no seu MySQL alguns produtos que "quebrem" a regra do limite:

SQL

```
INSERT INTO produtos (nome, quantidade, valor, categoria)
VALUES
("Creme Hidratante (Esgotado)", 0, 45.00, "Produto de beleza"),
("Sabonete (Superlotação)", 150, 5.00, "Produto de beleza");
```

2. Um Detalhe sobre a Matemática do Percentual

Na sua query, você criou uma coluna calculada usando a seguinte expressão:

- _____

```
(quantidade / 100) * 100 AS percentual
```

Matematicamente, dividir um valor por 100 e depois multiplicar por 100 devolve exatamente o número original. Ou seja, a sua coluna `percentual` vai apresentar rigorosamente o mesmo número que a coluna `quantidade`.

- **Se a regra de negócio for:** "A capacidade ideal do meu estoque é 100 unidades para qualquer produto", então o cálculo faz sentido (ex: 150 unidades = 150% da capacidade).
- **Se a capacidade máxima de cada produto for diferente:** Você precisaria calcular isso com base em uma nova coluna de "capacidade_maxima" ou mudar esse número fixo.

Você gostaria de transformar esse número "100" em uma variável dinâmica, permitindo que você envie o limite máximo de estoque pela própria URL no Postman na hora de testar?

4-)

O código da sua rota está **perfeito** e não vai causar nenhum erro no seu servidor Express. A sintaxe está correta e a lógica do banco de dados bate com a estrutura da tabela `produtos` que você me enviou antes.

Porém, se você testar isso no Postman agora mesmo, pode ter a falsa impressão de que deu errado.

Aqui está o que você precisa saber sobre essa rota específica:

1. O "Problema" dos Dados Atuais (Retorno Vazio)

A sua condição `WHERE quantidade <= 0 OR quantidade >= 100` está certíssima para buscar os produtos em situação crítica de estoque. O detalhe é que os produtos que você cadastrou no seu script anterior têm as seguintes quantidades: **12, 10 e 20**.

Como nenhum deles esgotou (`<= 0`) e nenhum está superlotado (`>= 100`), o Postman vai te retornar uma lista vazia `[]`. Isso não é um erro; o banco só está respondendo que, no momento, o estoque está dentro dos limites.

Como ver a rota funcionando de verdade:

Para testar no Postman e ver os dados aparecendo, insira no seu MySQL alguns produtos que "quebrem" a regra do limite:

SQL

```
INSERT INTO produtos (nome, quantidade, valor, categoria)
VALUES
("Creme Hidratante (Esgotado)", 0, 45.00, "Produto de beleza"),
("Sabonete (Superlotação)", 150, 5.00, "Produto de beleza");
```

2. Um Detalhe sobre a Matemática do Percentual

Na sua query, você criou uma coluna calculada usando a seguinte expressão:

```
(quantidade / 100) * 100 AS percentual
```

Matematicamente, dividir um valor por 100 e depois multiplicar por 100 devolve exatamente o número original. Ou seja, a sua coluna `percentual` vai apresentar rigorosamente o mesmo número que a coluna `quantidade`.

- **Se a regra de negócio for:** "A capacidade ideal do meu estoque é 100 unidades para qualquer produto", então o cálculo faz sentido (ex: 150 unidades = 150% da capacidade).
- **Se a capacidade máxima de cada produto for diferente:** Você precisaria calcular isso com base em uma nova coluna de "capacidade_maxima" ou mudar esse número fixo.

Você gostaria de transformar esse número "100" em uma variável dinâmica, permitindo que você envie o limite máximo de estoque pela própria URL no Postman na hora de testar?

5-)

Para resolver esse exercício, o seu código atual vai precisar de uma boa evolução. Atualmente, você está fazendo um `SELECT *`, mas o enunciado pede **cálculos matemáticos, agrupamentos e junção de tabelas** (já que o nome do produto está em uma tabela e as movimentações em outra).

Além disso, temos um detalhe importante: olhando o script do banco que você mandou antes, **a sua tabela `produtos` não tem o campo `unidade_medida`**. Vamos precisar resolver isso também.

Aqui está o roteiro completo do que fazer em cada ferramenta para entregar exatamente os 7 campos na ordem exigida.

Passo 1: MySQL Workbench (Ajustando o Banco)

Como precisamos relatar a "unidade de medida", precisamos adicionar essa coluna na tabela `produtos` e atualizar os dados de teste. Rode o seguinte script no seu Workbench:

```
USE `mydb`;

-- 1. Adiciona a coluna que estava faltando na tabela de produtos
ALTER TABLE produtos ADD COLUMN unidade_medida VARCHAR(20)
NOT NULL DEFAULT 'UN';

-- 2. Atualiza os produtos existentes com algumas unidades de medida para testar
UPDATE produtos SET unidade_medida = 'Litro' WHERE id = 1;
-- Bodysplash
UPDATE produtos SET unidade_medida = 'Unidade' WHERE id = 2;
-- Moletom
UPDATE produtos SET unidade_medida = 'Unidade' WHERE id = 3;
-- Batom

-- 3. Vamos garantir que temos movimentações reais de 'entrada' e 'saida' para testar as datas
-- (Apaga as antigas "sla", "abc" e insere dados válidos)
TRUNCATE TABLE movimentacoes;

INSERT INTO movimentacoes(datas, tipo, quantidade, produtos_id)
VALUES
('2026-05-10', 'entrada', 10, 1),
('2026-05-12', 'saida', 5, 1),
('2026-05-15', 'entrada', 20, 2),
('2026-05-20', 'saida', 10, 2),
('2026-05-22', 'saida', 4, 3);
```

Passo 2: VS Code (Construindo a Query Complexa)

Substitua a sua rota `/movimentacoes` por esta aqui. Note que usamos o `SUM(CASE WHEN...)` para separar o que é entrada e o que é saída e multiplicar pelo valor do

produto para obter o financeiro. Também corriji o nome da coluna de `date` para `datas`.

```
// 12- Listar movimentações de entrada e saída no período informado
router.get('/movimentacoes', (req, res) => {
  const { data_inicial, data_final } = req.query;

  if (!data_inicial || !data_final) {
    return res.status(400).json({ error: 'Por favor, informe a data_inicial e data_final na URL.' });
  }

  // Query robusta para trazer exatamente os 7 campos obrigatórios na ordem solicitada
  const querySQL = `
    SELECT
      p.nome AS nome_do_produto,
      p.unidade_medida,
      SUM(CASE WHEN m.tipo = 'entrada' THEN m.quantidade ELSE 0 END) AS total_de_entradas,
      SUM(CASE WHEN m.tipo = 'saida' THEN m.quantidade ELSE 0 END) AS total_de_saidas,
      SUM(CASE WHEN m.tipo = 'saida' THEN m.quantidade ELSE 0 END) AS saida_no_periodo,
      SUM(CASE WHEN m.tipo = 'entrada' THEN m.quantidade * p.valor ELSE 0 END) AS valor_total_financeiro_das_entradas,
      SUM(CASE WHEN m.tipo = 'saida' THEN m.quantidade * p.valor ELSE 0 END) AS valor_financeiro_das_saidas
    FROM produtos p
    INNER JOIN movimentacoes m ON p.id = m.produtos_id
    WHERE m.datas BETWEEN ? AND ?
    GROUP BY p.id, p.nome, p.unidade_medida;
  `;

  db.query(querySQL, [data_inicial, data_final], (err, results) => {
```

```

        if (err) {
            console.error('Erro no SQL:', err); // Ajuda vo
cê a debugar se algo falhar
            return res.status(500).json({ error: 'Erro ao b
uscar movimentações no período' });
        }
        res.json(results);
    });
});

```

Nota sobre a lógica: Como o relatório inteiro já é filtrado no período (`WHERE m.datas BETWEEN ...`), o "total de saídas" e a "saída no período" resultam no mesmo valor dentro desse escopo. Estructurei dessa forma para garantir que o seu JSON retorne as 7 chaves exatas que o enunciado pede.

Passo 3: Postman (Fazendo a Requisição Correta)

Como a rota utiliza `req.query`, você precisa passar as datas diretamente na URL usando o formato americano (`AAAA-MM-DD`).

1. Abra o Postman e crie uma nova aba de requisição.
2. Defina o método como **GET**.
3. Na barra de endereço, cole a URL configurando o período desejado (usando o exemplo de dados que cadastramos no Passo 1):

```

<http://localhost:3000/api/movimentacoes?data_inicial=2026-05-01&data_final=2026-05-31
>

```

1. Clique em **Send**.

O que você deve receber de resposta no Postman:

Um array de objetos organizados exatamente assim:

```

[
  {
    "nome_do_produto": "Bodysplash",
    "unidade_medida": "Litro",
    "total_de_entradas": 10,

```



```

    "total_de_saidas": 5,
    "saida_no_periodo": 5,
    "valor_total_financeiro_das_entradas": 200.00,
    "valor_financeiro_das_saidas": 100.00
  },
  {
    "nome_do_produto": "Moletom",
    "unidade_medida": "Unidade",
    "total_de_entradas": 20,
    "total_de_saidas": 10,
    "saida_no_periodo": 10,
    "valor_total_financeiro_das_entradas": 1000.00,
    "valor_financeiro_das_saidas": 500.00
  }
]

```

6-)

Erros em queries que envolvem cruzamento de tabelas (`JOIN`) e cálculos matemáticos são super comuns e, sendo sincero, bem irritantes de achar quando acontecem. Vamos resolver isso juntos, do zero, destrinchando cada etapa para você entender exatamente o que está acontecendo.

O primeiro segredo da programação quando algo dá errado é: **onde está o erro?**

- Se no Postman apareceu a mensagem `"Por favor, informe a data_inicial..."`, o erro está no formato da URL que você digitou no Postman.
- Se no Postman apareceu `"Erro ao buscar movimentações..."`, o erro está no banco de dados, e a mensagem real do erro (ex: coluna que não existe) apareceu no **Terminal do VS Code**.

Para limpar a barra e garantir que tudo funcione, vamos refazer o processo de forma ultra detalhada.

1. No MySQL Workbench (Garantindo a estrutura)

O erro mais provável é que o banco de dados não tenha a estrutura exata que a query precisa. Para não ter erro de "coluna inexistente", vamos apagar o banco

antigo e criar o novo com tudo o que precisamos (incluindo a coluna `unidade_medida` e dados com a palavra certa `'entrada'` e `'saida'`).

Copie todo o código abaixo, cole em uma aba nova no seu MySQL Workbench, **selecione tudo** e clique no raio para executar:

SQL

```
-- 1. Apaga o banco antigo se ele existir para começarmos do zero absoluto
DROP DATABASE IF EXISTS `mydb`;

-- 2. Cria o banco de dados limpo
CREATE DATABASE `mydb` DEFAULT CHARACTER SET utf8;
USE `mydb`;

-- 3. Cria a tabela de produtos (repare que adicionei a unidade_medida aqui)
CREATE TABLE `produtos` (
  `id` INT NOT NULL AUTO_INCREMENT,
  `nome` VARCHAR(255) NOT NULL,
  `valor` DECIMAL(10,2) NOT NULL,
  `quantidade` INT NOT NULL,
  `categoria` VARCHAR(255) NOT NULL,
  `unidade_medida` VARCHAR(50) NOT NULL, -- Coluna essencial para o seu exercício 12
  PRIMARY KEY (`id`)
) ENGINE = InnoDB;

-- 4. Cria a tabela de movimentações (repare que a coluna se chama 'datas')
CREATE TABLE `movimentacoes` (
  `id` INT NOT NULL AUTO_INCREMENT,
  `datas` DATE NOT NULL,
  `tipo` VARCHAR(10) NOT NULL, -- Onde vai entrar 'entrada' ou 'saida'
  `quantidade` INT NOT NULL,
  `produtos_id` INT NOT NULL,
  PRIMARY KEY (`id`),
  CONSTRAINT `fk_movimentacoes_produtos`
    FOREIGN KEY (`produtos_id`)
    REFERENCES `produtos` (`id`)
) ENGINE = InnoDB;

-- 5. Inserindo dados de teste válidos para o ano de 2026
INSERT INTO produtos (nome, quantidade, valor, categoria, unidade_medida) VALUES
("Bodysplash", 12, 20.00, "Produto de beleza", "Litro"),
("Moletom", 10, 50.00, "Roupa", "Unidade"),
("Batom", 20, 10.00, "Produto de beleza", "Unidade");

INSERT INTO movimentacoes (datas, tipo, quantidade, produtos_id) VALUES
('2026-05-10', 'entrada', 10, 1), -- 10 Bodysplash entrando
('2026-05-12', 'saida', 5, 1),   -- 5 Bodysplash saindo
('2026-05-15', 'entrada', 20, 2), -- 20 Moletons entrando
('2026-05-20', 'saida', 10, 2),  -- 10 Moletons saindo
('2026-05-22', 'saida', 4, 3);   -- 4 Batons saindo
```

2. No VS Code (O Código Explicado)

No seu arquivo de rotas do VS Code, substitua a rota antiga por esta. Eu adicionei comentários explicando o que cada linha da query SQL faz:

JavaScript

```
// 12- Listar movimentações de entrada e saída no período informado
router.get('/movimentacoes', (req, res) => {
  // 1. Pegamos as duas datas que o usuário vai digitar na URL do Postman
  const { data_inicial, data_final } = req.query;

  // 2. Se o usuário esquecer de passar alguma das datas, o sistema para aqui e avisa
  if (!data_inicial || !data_final) {
    return res.status(400).json({ error: 'Por favor, informe data_inicial e data_final na URL.' });
  }

  // 3. A Query SQL detalhada para cumprir os 7 campos obrigatórios
  const querySQL = `
    SELECT
      p.nome AS nome_do_produto,           -- [Campo 1] Nome do produto (Vem da tabela produtos)
      p.unidade_medida,                   -- [Campo 2] Unidade de medida (Vem da tabela produtos)
      SUM(CASE WHEN m.tipo = 'entrada' THEN m.quantidade ELSE 0 END) AS total_de_entradas, -- [Campo 3] Soma qnd for entrada
      SUM(CASE WHEN m.tipo = 'saida' THEN m.quantidade ELSE 0 END) AS total_de_saidas, -- [Campo 4] Soma qnd for saida
      SUM(CASE WHEN m.tipo = 'saida' THEN m.quantidade ELSE 0 END) AS saida_no_periodo, -- [Campo 5] Saida no periodo
      SUM(CASE WHEN m.tipo = 'entrada' THEN m.quantidade * p.valor ELSE 0 END) AS valor_total_financeiro_das_entradas, -- [Campo 6] Qtd * Valor
      SUM(CASE WHEN m.tipo = 'saida' THEN m.quantidade * p.valor ELSE 0 END) AS valor_financeiro_das_saidas -- [Campo 7] Qtd * Valor
    FROM produtos p
    INNER JOIN movimentacoes m ON p.id = m.produtos_id -- Junta as duas tabelas usando a chave estrangeira
    WHERE m.datas BETWEEN ? AND ? -- Filtra apenas as datas que estão entre o que foi digitado
    GROUP BY p.id, p.nome, p.unidade_medida; -- Agrupa por produto para não repetir o mesmo nome várias vezes
  `;

  // 4. Executa a query substituindo os '?' pelas datas recebidas
  db.query(querySQL, [data_inicial, data_final], (err, results) => {
    if (err) {
      // SE DEU ERRO, ESSA LINHA VAI PRINTAR O ERRO REAL NO TERMINAL DO VS CODE!
      console.error("ERRO REAL DO BANCO DE DADOS ->", err);
      return res.status(500).json({ error: 'Erro ao buscar movimentações no período' });
    }
  });
}
```

```
// 5. Devolve os dados bonitinhos em formato JSON
res.json(results);
});
});
```

Atenção: Depois de salvar o arquivo no VS Code, você **precisa reiniciar o servidor**. Vá no terminal do VS Code, aperte **Ctrl + C** para derrubar o servidor antigo e rode **node seu_arquivo.js** novamente.

3. No Postman (Como fazer a chamada)

O erro comum aqui é a forma de passar os parâmetros. Como usamos **req.query**, os dados vão na própria URL após um ponto de interrogação **?**.

1. Abra o Postman e mude o método para **GET**.
2. No campo de URL, cole exatamente isso (considerando as datas dos dados que inserimos no passo 1):Plaintext

```
http://localhost:3000/api/movimentacoes?data_inicial=2026-05-01&data_final=2026-05-31
```

3. Se você olhar logo abaixo da barra de URL do Postman, na aba **Params**, você verá que ele preencheu automaticamente uma tabela com:

- **Key:** **data_inicial** | **Value:** **2026-05-01**
- **Key:** **data_final** | **Value:** **2026-05-31**

4. Clique em **Send**.

Se você seguiu os passos acima, o banco de dados vai processar a matemática e te devolver uma resposta limpa, mostrando as somas e os valores financeiros calculados automaticamente para cada produto dentro do mês de maio de 2026.

Caso ainda apareça alguma mensagem de erro, dê uma olhada no terminal do VS Code: qual é o texto exato que aparece logo após a frase **ERRO REAL DO BANCO DE DADOS -> ?**

7-)

Quando o Postman dá erro ao enviar o link, geralmente é um problema de "endereço errado" ou o servidor do VS Code está desligado. Como "deu erro" pode ser muita coisa, vamos identificar qual é o seu erro pelo que aparece na tela do Postman.

Olhe para a resposta do Postman e veja em qual desses 4 casos você se encaixa:

Caso 1: O Postman diz "Could not get any response" ou "ECONNREFUSED"

Isso significa que o Postman tentou bater na porta `3000` (ou a porta que você escolheu), mas **não encontrou ninguém em casa**. O seu servidor Node.js está desligado.

- **Como resolver:** 1. Vá no VS Code.
2. Olhe o terminal embaixo. Ele está limpo ou parado?
3. Digite o comando para ligar o servidor (ex: `node index.js` ou `npm start`).
4. Ele precisa mostrar aquela mensagem: *"Servidor rodando na porta 3000"*. Não feche esse terminal!

Caso 2: Status 404 Not Found

O servidor está ligado, mas você errou o caminho da URL. O Express recebeu a chamada, mas disse: *"Não tenho nenhuma rota com esse nome"*.

- **Como resolver:** Verifique como você configurou as rotas no seu arquivo principal (geralmente `index.js` ou `app.js`).
 - Se você usou `app.use('/api', router);` , a URL obrigatoriamente tem que ter o `/api` :
`http://localhost:3000/api/movimentacoes?data_inicial=2026-05-01&data_final=2026-05-31`
 - Se você usou `app.use('/', router);` ou apenas `app.use(router);` , a URL **não** tem o `/api` :
`http://localhost:3000/movimentacoes?data_inicial=2026-05-01&data_final=2026-05-31`

Caso 3: Status 400 Bad Request (Aparece a mensagem "Por favor, informe data_inicial...")

O servidor está ligado, achou a rota, mas as datas não chegaram até o código do VS Code.

- **Como resolver:** Pare de tentar digitar o link gigante na barra de endereço e use os campos do Postman:
 1. No Postman, digite apenas `http://localhost:3000/api/movimentacoes` (ou sem o `/api`).
 2. Logo abaixo da URL, clique na aba **Params** (Parâmetros).
 3. Na tabela que abrir, preencha exatamente assim:
 - Na coluna **Key**, digite: `data_inicial` | Na coluna **Value**, digite: `2026-05-01`
 - Na linha de baixo, na coluna **Key**, digite: `data_final` | Na coluna **Value**, digite: `2026-05-31`
 4. Note que o Postman vai montando o link sozinho lá em cima. Clique em **Send**.

Caso 4: Status 500 Internal Server Error

A rota foi encontrada, o Postman enviou as datas, mas o código bateu no MySQL e o banco rejeitou a query por algum motivo (nome de coluna errado, banco desconectado, etc).

- **Como resolver:** Deixe o Postman de lado por um segundo, abra o VS Code e olhe para o **Terminal**. Como nós colocamos um `console.error` no código, o erro real do MySQL vai estar escrito lá em letras vermelhas ou brancas.

Qual é o **Status** (ex: 404, 500) ou a mensagem exata que aparece na metade de baixo da tela do seu Postman quando você clica em Send?

8-)

Ótimas notícias! O seu servidor está ligado, a rota foi encontrada e o seu código está **funcionando perfeitamente**.

Esse erro `400 Bad Request` com a mensagem *"Por favor, informe a data_inicial..."* foi a validação que nós criamos justamente para travar a requisição caso as datas não fossem enviadas.

Olhando para a sua foto, repare que a tabela de **Query Params** (no meio da tela) está completamente vazia. O Postman mandou o link limpo, e o VS Code

barrou por falta de informação.

Como resolver isso no Postman

Você tem duas formas muito simples de colocar as datas para o teste funcionar:

Opção 1: Preenchendo a tabela (Mais fácil)

Logo abaixo da barra de endereços, onde diz **Query Params**, preencha a tabela exatamente assim:

1. Na coluna **Key**, digite: `data_inicial`
2. Na coluna **Value** ao lado, digite: `2026-05-01`
3. Na linha de baixo, na coluna **Key**, digite: `data_final`
4. Na coluna **Value** ao lado, digite: `2026-05-31`

(Você vai notar que, conforme digita, o Postman vai alterando o link lá de cima sozinho).

Opção 2: Colando o link completo direto na barra

Se preferir, pode simplesmente apagar o link atual da barra de endereços do Postman e colar ele completo com os parâmetros de data inclusos:

Plaintext

```
http://localhost:3000/produtos/movimentacoes?data_inicial=2026-05-01&data_final=2026-05-31
```

Depois de fazer um dos dois passos acima, clique no botão azul **Send** novamente e o banco de dados vai processar e listar todos os 7 campos calculados na tela.

9-)

Calma, não desista! Quando passamos da fase do erro 400, o Postman tenta avançar, mas qualquer detalhe desalinhado entre o código e o banco de dados vai travar o resultado.

Como agora você colocou os parâmetros, precisamos descobrir o que apareceu na tela. Vamos direto aos **3 cenários** do que pode ter acontecido agora e como resolver cada um:

Cenário 1: Continuou dando o erro 400 (Bad Request)

Se a mensagem *"Por favor, informe a data_inicial..."* continuou aparecendo, o VS Code ainda não está conseguindo ler as datas.

- **O motivo:** Um erro de digitação bobo no Postman.
- **Como corrigir:** Verifique se você escreveu exatamente `data_inicial` e `data_final` (com o underline `_`). Se escrever `dataInicial` (com o I maiúsculo) ou `data-inicial` (com hífen), o código não vai reconhecer.

Cenário 2: Mudou para o erro 500 (Internal Server Error)

Se apareceu **500**, significa que o Postman enviou as datas certo, mas o MySQL rejeitou a nossa query complexa.

- **O motivo:** Provavelmente a sua tabela no MySQL Workbench não tem a coluna `unidade_medida` ou a tabela de relacionamentos está com algum nome diferente.
- **Como descobrir o erro real:** Deixe o Postman de lado e olhe para o **Terminal do VS Code** (aquela parte preta embaixo onde o servidor roda). Como nós colocamos um `console.error` no código, o MySQL escreveu em texto plano o motivo exato de ter falhado.

Cenário 3: Deu Status 200 (OK), mas retornou um array vazio

`[]`

Se apareceu apenas `[]` na tela, a rota funcionou perfeitamente e o banco aceitou a query, mas ele não encontrou nenhuma linha que batesse com o que você pediu.

- **O motivo:** 1. As datas que você digitou no Postman não possuem nenhuma movimentação correspondente no banco.
2. O banco de dados está vazio (você esqueceu de rodar os `INSERTs` de teste no MySQL Workbench).
- **Como corrigir:** Certifique-se de que rodou os `INSERTs` que te mandei na mensagem anterior e que a data no Postman cubra o período correto (ex: `2026-05-01` até `2026-05-31`).

Para eu te dar a resposta exata e matarmos esse problema agora: **Qual é o Status que aparece no Postman agora (ex: 400, 500 ou 200) ou qual erro apareceu no terminal do seu VS Code?**

10-)

Para o **Exercício 13**, você está enfrentando exatamente o mesmo desafio do anterior: o código atual é apenas um `SELECT *` simples que vai quebrar porque a coluna `date` não existe (chama-se `datas`) e os 3 campos obrigatórios calculados não foram feitos.

Como o objetivo aqui é listar os produtos com **maior volume de saída**, precisamos filtrar apenas as movimentações do tipo `'saida'`, somar as quantidades, calcular o valor financeiro total dessas saídas, agrupar por produto e ordenar do maior para o menor (`DESC`).

Aqui está tudo o que você precisa fazer no VS Code e no Postman para deixar essa rota funcionando perfeitamente, utilizando o mesmo banco de dados que estruturamos no passo anterior.

1. No VS Code (Atualizando a rota `/maiores-saidas`)

Substitua todo o código da sua rota `/maiores-saidas` por este aqui. Note que incluímos a junção de tabelas (`INNER JOIN`), os filtros corretos e a ordenação por volume:

JavaScript

```
// 13- listar os produtos com maior volume de saida no periodo informado
router.get('/maiores-saidas', (req, res) => {
  const { data_inicial, data_final } = req.query;

  // Validação para garantir que as datas foram enviadas na URL
  if (!data_inicial || !data_final) {
    return res.status(400).json({ error: 'Por favor, informe data_inicial e data_final na URL.' });
  }

  // Query SQL estruturada para trazer os 3 campos obrigatórios na ordem exata
  const querySQL = `
    SELECT
      p.nome AS nome_do_produto,
      SUM(m.quantidade) AS quantidade_total_de_saidas,
      SUM(m.quantidade * p.valor) AS valor_total_financeiro_das_saidas
    FROM produtos p
    INNER JOIN movimentacoes m ON p.id = m.produto_id
    WHERE m.tipo = 'saida'
    AND m.data BETWEEN ? AND ?
    GROUP BY p.id, p.nome
    ORDER BY quantidade_total_de_saidas DESC
  `;

  res.json(res.status(200).json({ data: await db.query(querySQL, [data_inicial, data_final]) })));
});
```

```

    mpo 3]
        FROM produtos p
        INNER JOIN movimentacoes m ON p.id = m.produtos_id
        WHERE m.tipo = 'saida' AND m.datas BETWEEN ? AND ? -- Filtra apenas as SAÍDAS
dentro do período
        GROUP BY p.id, p.nome
        ORDER BY quantidade_total_de_saidas DESC;          -- Garante os de maior vol
ume primeiro
    `;

    db.query(querySQL, [data_inicial, data_final], (err, results) => {
        if (err) {
            console.error("ERRO NO BANCO (ROTA 13) ->", err); // Printa o erro real no
terminal do VS Code
            return res.status(500).json({ error: 'Erro ao buscar produtos com maiores
saídas' });
        }
        res.json(results); // Retorna a lista ordenada para o Postman
    });
});

module.exports = router;

```

Lembrete Obrigatório: Salve o arquivo no VS Code, vá até o terminal rodando o Node, aperte **Ctrl + C** para desligar o servidor e ligue-o novamente com **node seu_arquivo.js** para aplicar as alterações.

2. No Postman (Testando a rota)

Para testar sem cair no erro **400 Bad Request**, precisamos enviar as datas formatadas corretamente através dos parâmetros da URL.

1. Abra uma nova aba no Postman e configure o método como **GET**.
2. Na barra de endereços, cole a URL apontando para a nova rota com os parâmetros de maio de 2026:Plaintext

```

http://localhost:3000/produtos/maiores-saidas?data_inicial=2026-05-01&data_final=2026-05-31

```

3. Se preferir usar a tabela **Query Params** do Postman (abaixo da URL), configure assim:

- **Key:** `data_inicial` | **Value:** `2026-05-01`
- **Key:** `data_final` | **Value:** `2026-05-31`

4. Clique em **Send**.

O Resultado Esperado

O Postman deve retornar um Status **200 OK** com o JSON trazendo os produtos que tiveram saídas no mês de maio, ordenados do que mais vendeu para o que menos vendeu:

JSON

```
[
  {
    "nome_do_produto": "Moletom",
    "quantidade_total_de_saidas": 10,
    "valor_total_financeiro_das_saidas": 500.00
  },
  {
    "nome_do_produto": "Bodysplash",
    "quantidade_total_de_saidas": 5,
    "valor_total_financeiro_das_saidas": 100.00
  },
  {
    "nome_do_produto": "Batom",
    "quantidade_total_de_saidas": 4,
    "valor_total_financeiro_das_saidas": 40.00
  }
]
```

11-)

Vamos descobrir o que aconteceu. Quando "dá erro", a chave para resolver em dois segundos está em olhar o **Status Code** (o número que o Postman mostra no canto superior direito da resposta, perto do tempo de execução).

Aqui está o mapa para consertarmos isso agora, dependendo do número que apareceu na sua tela:

Se o status for 404 Not Found 🔍

O Express não encontrou essa rota. Olhando o seu print anterior, você usava o caminho `/produtos/movimentacoes`.

- **O erro:** Provavelmente você digitou apenas `/maiores-saidas` ou esqueceu do prefixo.
- **A solução:** Altere a URL no Postman para garantir que ela tenha o `/produtos/`. O link correto deve ser exatamente este: Plaintext

```
http://localhost:3000/produtos/maiores-saidas?data_inicial=2026-05-01&data_final=2026-05-31
```

Se o status for 500 Internal Server Error 💥

O Postman achou a rota, mas o MySQL rejeitou a query. Como mudamos os cálculos e juntamos tabelas (`JOIN`), o banco pode estar reclamando de algum nome.

- **O erro:** Algum nome de coluna ou tabela está desalinhado no banco.
- **A solução:** Deixe o Postman de lado por um segundo, abra o VS Code e olhe para o seu **Terminal** (a parte de baixo). Como deixamos o `console.error` ativado, vai ter uma mensagem lá dizendo exatamente o que o MySQL não gostou.

Exemplo comum: Se disser `Table 'mydb.movimentacoes' doesn't exist`, verifique se no seu banco ela não foi criada como `movimentacao` (no singular).

Se o status for 400 Bad Request ⚠️

A validação barrou a entrada de dados de novo.

- **O erro:** As datas não foram enviadas ou os nomes das chaves ficaram diferentes do código.
- **A solução:** Vá na aba **Params** do Postman e certifique-se de que escreveu `data_inicial` e `data_final` tudo em letras minúsculas e com o sinal de underline (`_`).

O erro clássico: Esquecer de reiniciar o servidor 🔄

Se você apenas colou o código no VS Code e tentou rodar no Postman direto, vai dar erro porque o Node.js ainda está executando o seu código antigo.

1. Vá no terminal do VS Code.
2. Aperte `ctrl + c` para derrubar o servidor.
3. Digite o comando para ligar novamente (ex: `node index.js`).
4. Tente o botão **Send** no Postman outra vez.

Para irmos direto ao ponto e resolver: **Qual é o número do Status (ex: 404, 500) que apareceu no seu Postman dessa vez?**